

College Club & Event Voting Portal

Deep Research, System Design & Frontend Architecture

Final technology: React.js + Vite + Tailwind CSS + Axios + MSW Mock Backend + RBAC + optional Framer Motion. All authored React source modules use `.jsx`; no TypeScript.

1. Executive Summary

The portal supports two roles. Student Voters browse candidate profiles and manifestos, cast exactly one vote per category, and receive a confirmation receipt. Election Managers add/edit candidates, toggle voting Open or Closed, and view live tally bars. The architecture is frontend-first but keeps a realistic API boundary through Axios and MSW.

The main design principles are clarity for voters, strict role separation and vote-integrity rules. RBAC is implemented at route, UI-action and mock API levels rather than only hiding buttons.

2. Assessment-Aligned Technical Direction

The uploaded guidelines require React.js or Next.js, Tailwind CSS, Axios, a lightweight mock backend, responsive behavior and RBAC. This design selects React.js with Vite. Framer Motion, automated E2E testing and caching are optional enhancements.

3. RBAC Permission Matrix

Capability	Student Voter	Election Manager
View candidate profiles	Allow	Allow
View manifestos	Allow	Allow
Cast vote	Allow once per category when Open	Deny
View own receipt	Allow	Optional audit access if required
Add/Edit candidate	Deny	Allow
Toggle Open/Closed	Deny	Allow
View live tally	Deny in core brief	Allow

4. Permission Architecture

Use `permissions.jsx` for centralized role permissions, `ProtectedRoute.jsx` for page access, `PermissionGate.jsx` for actions, and MSW handlers for mutation validation. Student permissions: `VIEW_CANDIDATES`, `VIEW_MANIFESTO`, `CAST_VOTE`, `VIEW_RECEIPT`. Manager permissions: `ADD_CANDIDATE`, `EDIT_CANDIDATE`, `TOGGLE_VOTING`, `VIEW_TALLY`.

5. Student Voter UX Design

Dashboard: election title, Open/Closed badge, categories and per-category state: Available, Voted or Closed.

Candidate discovery: cards with image/avatar, name, category, short introduction and View Profile. Add lightweight search/filtering.

Candidate detail: profile, highlights and manifesto. Optional comparison should remain factual and must not label a system-generated 'best candidate'.

Vote review: selected category and candidate are shown before confirmation. Axios sends the request; MSW validates role, election state and duplicate voting.

Receipt: generated receipt ID, category, selected candidate and submission time. The category then becomes Voted.

6. Election Manager UX Design

Dashboard: election status, categories, candidate count and total votes. Main actions: Add Candidate and Open/Close Voting.

Candidate management: reusable CandidateForm.jsx with name, category, image URL, bio, manifesto and highlights.

Voting control: toggle → confirmation modal → Axios mutation → success toast → updated status.

Live tally: category selector plus animated bars. In this mock application, refresh after vote mutations or poll; WebSockets are not required.

7. Core Business Rules

1. Only Student Voters may vote. 2. Voting must be Open. 3. One vote per student per category. 4. Managers cannot vote. 5. Only Managers can change candidates or election status. 6. Unauthorized route access must redirect or show a warning.

The key integrity constraint is **unique(studentId, categoryId)**. A student can vote in multiple categories but never twice within the same category.

8. User Flows

Student: Role Switcher → Dashboard → Category → Candidates → Profile/Manifesto → Select → Review → Confirm → Validation → Receipt.

Manager: Role Switcher → Dashboard → Add/Edit Candidate → Save → Open/Close Voting → Confirm → Live Tally.

9. Route Architecture

Shared: /, /role-selection, /unauthorized

Student: /student/dashboard, /student/categories, /student/candidate/:id, /student/vote/:categoryId, /student/receipt/:receiptId

Manager: /manager/dashboard, /manager/candidates, /manager/candidates/new, /manager/candidates/:id/edit, /manager/tally

10. JSX Project Structure

```
college-voting-portal/
├── src/
│   ├── api/axiosClient.jsx, electionApi.jsx, candidateApi.jsx, voteApi.jsx, managerApi.jsx
│   ├── components/common/, student/, manager/
│   ├── context/AuthContext.jsx
│   ├── hooks/useRole.jsx
│   ├── layouts/StudentLayout.jsx, ManagerLayout.jsx
│   ├── mocks/browser.jsx, handlers.jsx, data.jsx
│   ├── pages/student/*.jsx
│   ├── pages/manager/*.jsx
│   ├── routes/AppRoutes.jsx, ProtectedRoute.jsx
│   ├── utils/permissions.jsx
│   ├── App.jsx
│   ├── main.jsx
│   ├── package.json
│   └── vite.config.js
```

package.json and vite.config.js remain normal project configuration files; React application source files are authored as .jsx.

11. Data Model

Entity	Important fields
User	id, name, role
Election	id, title, status
Category	id, electionId, name
Candidate	id, categoryId, name, bio, manifesto, imageUrl
Vote	id, studentId, categoryId, candidateId, createdAt
Receipt	id, voteId, issuedAt

12. Axios API Contract

Method	Endpoint	Purpose
GET	/api/elections/:id	Election, categories and status
GET	/api/categories/:id/candidates	Candidates for category

Method	Endpoint	Purpose
GET	/api/candidates/:id	Profile and manifesto
POST	/api/votes	Validate and create vote
GET	/api/receipts/:id	Confirmation receipt
POST	/api/manager/candidates	Create candidate
PATCH	/api/manager/candidates/:id	Edit candidate
PATCH	/api/manager/elections/:id/status	Open/close voting
GET	/api/manager/elections/:id/tally	Vote totals

Recommended responses: success = { success: true, data: ... }; error = { success: false, error: { code, message, details } }.
Suggested codes: FORBIDDEN, ELECTION_CLOSED, ALREADY_VOTED, VALIDATION_ERROR and NOT_FOUND.

13. MSW Mock Backend Design

Axios performs all requests. MSW intercepts them through browser.jsx and handlers.jsx. data.jsx stores seed users, elections, categories, candidates and votes. Handlers should mutate in-memory state so new votes affect tally results during the session.

Important simulations: duplicate vote → 409, closed election → rejection, unauthorized manager mutation → 403, invalid input → 400 and missing resource → 404.

14. Tailwind CSS UI System

Create reusable components: AppShell, Navbar, Sidebar, StatusBadge, CandidateCard, ConfirmModal, PermissionGate, VoteProgress, ReceiptCard, CandidateForm and TallyBar.

Mobile: single-column cards and sticky vote action. **Tablet:** two-column grids and collapsible navigation. **Desktop:** sidebar, summary cards and wider management tables.

15. Framer Motion and Accessibility

Use motion for page transitions, candidate hover, modal entry, receipt success and tally bars. Respect reduced-motion preferences.

All controls must be keyboard accessible. Use semantic buttons, visible focus states, proper labels and focus-aware modals. Do not communicate Open/Closed or errors only through color.

16. Security and Vote Integrity

The role switcher is a mock authentication mechanism, but the architecture should treat authorization as authoritative at the API boundary. Use deny-by-default, least privilege, centralized permissions, election-state validation and unique(studentId, categoryId).

17. Implementation Roadmap

1. Foundation — React/Vite, Tailwind, responsive app shell and folder structure.
2. RBAC — role switcher, AuthContext.jsx, permissions.jsx and ProtectedRoute.jsx.
3. Mock API — Axios client, MSW setup, seed data and request handlers.
4. Student flow — candidates, details, vote confirmation and receipt.
5. Manager flow — candidate CRUD, election state and live tally.
6. Quality — loading, error, empty, unauthorized and mobile states.
7. Bonus — Framer Motion and E2E tests.
8. Delivery — README, screenshots, conventional commits and deployment.

18. Critical Test Cases

1. Student can browse profiles and manifestos.
2. Student can vote once in Category A and once in Category B.
3. Duplicate vote in the same category is rejected.
4. Closed election prevents voting.
5. Student cannot access manager routes.
6. Manager mutation is rejected when called by Student role.
7. Manager can add and edit candidates.
8. Manager can toggle Open/Closed.
9. Tally changes after a valid vote.
10. Keyboard and modal focus behavior work.
11. Mobile layout has no horizontal overflow.

19. Final Recommendation

Build one polished application with two distinct experiences. Student flow: **discover** → **evaluate** → **confirm** → **receipt**. Manager flow: **configure** → **control election state** → **monitor tally**. Apply RBAC to routes, actions and MSW API mutations.

Final stack: **React.js + Vite + JavaScript/JSX + Tailwind CSS + Axios + MSW + React Router DOM + Context API + RBAC + Framer Motion**. All authored React source modules use **.jsx**.

Source basis

The uploaded assessment guidelines provide the project constraints: modern frontend standards, React.js/Next.js, Tailwind CSS, Axios, lightweight mock backend options, RBAC, responsive design, Git hygiene, deployment and optional Framer Motion/testing. Additional security and accessibility recommendations are design guidance added to strengthen the implementation.